Attention Mechanism Deep Dive

Naeemullah Khan
naeemullah.khan@kaust.edu.sa

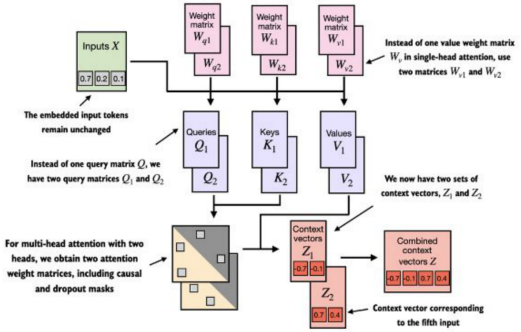
جامعة الملك عبدالله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

August 9, 2026

Table of Contents

Attention Mechanism



Source: after Sebastian Raschka, *Build a Large Language Model (From Scratch)*, Manning, 2024, Ch. 3 (multi-head attention).

A query is scored against every key, the scores are normalized into weights, and the output is the weighted sum of the values. Running several such heads in parallel gives multi-head attention, covered later.

Motivation

- Sequence-to-sequence models struggle with **long-range dependencies**, making it difficult to capture context over lengthy inputs.
- Attention mechanisms address this by allowing the model to **selectively focus** on relevant parts of the input sequence during processing.
- They form the **backbone** of modern architectures such as **Transformer models** (e.g., BERT, GPT, ViT).
- Attention underpins tasks such as **translation**, **summarization**, and **image captioning**.

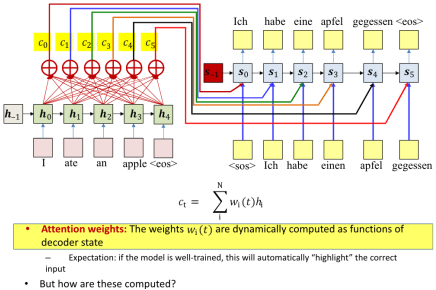
Example: In machine translation, rather than encoding an entire sentence into a single fixed-size vector, attention enables the model to dynamically focus on relevant source words when generating each target word.

Learning Outcomes

After this session, you should be able to:

- Understand how **alignment functions** operate.
- Comprehend **QKV (Query, Key, Value)** mechanism in attention.
- Learn how attention is used in **decoding sequences**.
- Recognize the **limitations** of attention mechanisms and the trade-offs behind them.
- Get familiar with **modern attention research**.

The Attention Mechanism (Recap)



Source: after CMU 11-785 (Spring 2024), Lecture 18: Sequence to Sequence Models & Attention, slide 28.

- The decoder builds a fresh context vector at every step: $c_t = \sum_i w_i(t)\, h_i$, a weighted sum of all encoder states, not the hidden state of any single input word.
- The weights $w_i(t)$ are computed dynamically from the previous decoder state s_{t-1}, are non-negative, and sum to one (softmax over scores).
- The scores behind these weights come from an alignment function, covered next.

Alignment Function

- The alignment function is the component of attention that determines how much focus to place on different parts of the input sequence.
- It computes a score for each input token relative to the current output token being generated.
- The scores are then normalized (often using softmax) to create a probability distribution, indicating the relative importance of each input token.

Importance of Alignment Function

The choice of alignment function can significantly impact the model's performance, especially in tasks with long sequences or complex dependencies.

Example: In a translation task, the alignment function helps the model determine which words in the source language should be emphasized when generating each word in the target language. For instance, when translating "The cat sat on the mat," the model might focus more on "cat" when generating "gato" in Spanish, while still considering "sat" and "on" to maintain the context.

Types of Alignment Functions

Here **q** is the query formed from the decoder state and **k** the key formed from an encoder state; both are made explicit in the next section.

- **Dot Product**:

$$\text{score} = \mathbf{q}^\top \mathbf{k}$$

- **Scaled Dot Product**:

$$\text{score} = \frac{\mathbf{q}^\top \mathbf{k}}{\sqrt{d_k}}$$

- **Additive (Bahdanau) Attention**:

$$\text{score} = \mathbf{v}^\top \tanh(W_1 h_i + W_2 s_{t-1})$$

- **General**:

$$\text{score} = \mathbf{q}^\top W_a \mathbf{k}$$

Choosing an Alignment Function

- **Dot product** adds no parameters and reduces to a single matrix product, so it is the cheapest, but it requires $\mathbf{q}$ and $\mathbf{k}$ to have the same dimension.
- **Scaling by** $\sqrt{d_k}$ matters because, for independent components with zero mean and unit variance, $\mathbf{q}^\top \mathbf{k}$ has variance d_k. Left unscaled, the scores grow with dimension and push the softmax into a region where its gradients are vanishingly small.
- **Additive attention** passes the pair through a hidden layer, so query and key need not share a dimension, at the cost of an extra weight matrix and a nonlinearity per score.
- **General** sits between the two: one learned matrix W_a bridges mismatched dimensions while keeping the score a single product.

Query, Key, Value (QKV)

- **Query:** What you want (the search)
- **Key:** Index of content (where to look)
- **Value:** Actual content (what you get)

Intuition

Think of QKV like a search engine:

- **Query (Q):** Comes from the current decoder state (or token)
- **Key (K)**, **Value (V):** Come from the encoder (or input sequence)

QKV – Formulation

Given:

- Input sequence $\mathbf{X}$

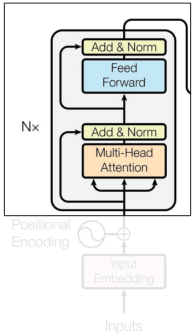
Linear projections:

$$\mathbf{Q} = \mathbf{X}W_Q$$
$$\mathbf{K} = \mathbf{X}W_K$$
$$\mathbf{V} = \mathbf{X}W_V$$

Attention: score $\mathbf{Q}$ against $\mathbf{K}$ with the scaled dot-product alignment, softmax the scores, and take the weighted sum of $\mathbf{V}$.

Query, Key, Value (QKV)

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

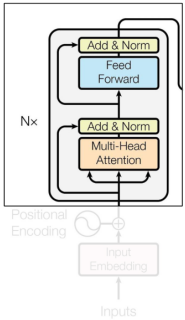


Source: CMU 11-785 (Spring 2024), Lecture 19: Transformers and Newer Architectures, slide 33.

Query, Key, Value (QKV)

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

- Query
- Key
- Value



Source: CMU 11-785 (Spring 2024), Lecture 19: Transformers and Newer Architectures, slide 34.

Query, Key, Value (QKV)

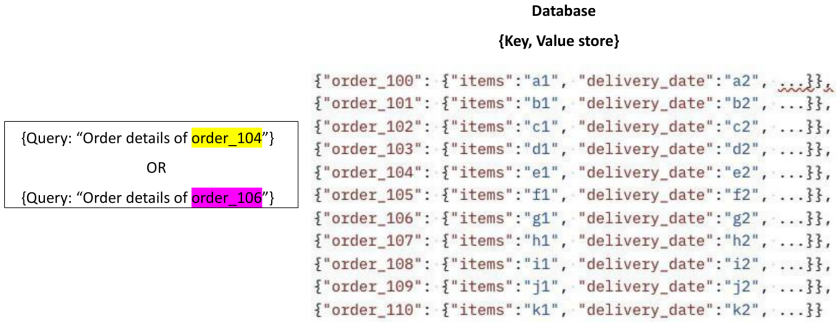
Database

{Key, Value store}

```
{"order_100": {"items":"a1", "delivery_date":"a2", ...}},
{"order_101": {"items":"b1", "delivery_date":"b2", ...}},
{"order_102": {"items":"c1", "delivery_date":"c2", ...}},
{"order_103": {"items":"d1", "delivery_date":"d2", ...}},
{"order_104": {"items":"e1", "delivery_date":"e2", ...}},
{"order_105": {"items":"f1", "delivery_date":"f2", ...}},
{"order_106": {"items":"g1", "delivery_date":"g2", ...}},
{"order_107": {"items":"h1", "delivery_date":"h2", ...}},
{"order_108": {"items":"i1", "delivery_date":"i2", ...}},
{"order_109": {"items":"j1", "delivery_date":"j2", ...}},
{"order_110": {"items":"k1", "delivery_date":"k2", ...}}
```

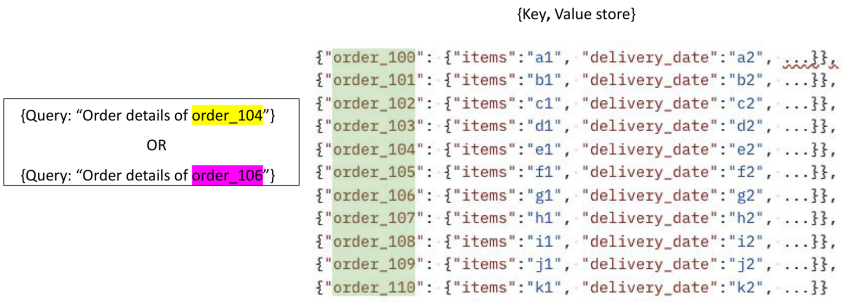
Source: CMU 11-785 (Spring 2024), Lecture 19: Transformers and Newer Architectures, slide 35.

Query, Key, Value (QKV)



Source: CMU 11-785 (Spring 2024), Lecture 19: Transformers and Newer Architectures, slide 36.

Query, Key, Value (QKV)



Source: CMU 11-785 (Spring 2024), Lecture 19: Transformers and Newer Architectures, slide 37.

Query, Key, Value (QKV)

{Query: “Order details of order_104”}

OR

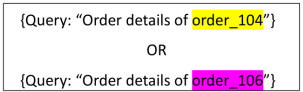
{Query: “Order details of order_106”}

{Key, Value store}

```
{"order_100": {"items":"a1", "delivery_date":"a2", ...}},
{"order_101": {"items":"b1", "delivery_date":"b2", ...}},
{"order_102": {"items":"c1", "delivery_date":"c2", ...}},
{"order_103": {"items":"d1", "delivery_date":"d2", ...}},
{"order_104": {"items":"e1", "delivery_date":"e2", ...}},
{"order_105": {"items":"f1", "delivery_date":"f2", ...}},
{"order_106": {"items":"g1", "delivery_date":"g2", ...}},
{"order_107": {"items":"h1", "delivery_date":"h2", ...}},
{"order_108": {"items":"i1", "delivery_date":"i2", ...}},
{"order_109": {"items":"j1", "delivery_date":"j2", ...}},
{"order_110": {"items":"k1", "delivery_date":"k2", ...}}
```

Source: CMU 11-785 (Spring 2024), Lecture 19: Transformers and Newer Architectures, slide 38.

Query, Key, Value (QKV)

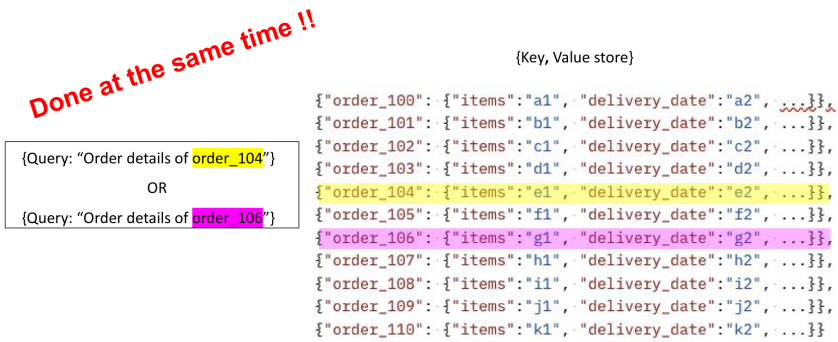


{Key, Value store}

```
{"order_100": {"items":"a1", "delivery_date":"a2", ...}},
{"order_101": {"items":"b1", "delivery_date":"b2", ...}},
{"order_102": {"items":"c1", "delivery_date":"c2", ...}},
{"order_103": {"items":"d1", "delivery_date":"d2", ...}},
{"order_104": {"items":"e1", "delivery_date":"e2", ...}},
{"order_105": {"items":"f1", "delivery_date":"f2", ...}},
{"order_106": {"items":"g1", "delivery_date":"g2", ...}},
{"order_107": {"items":"h1", "delivery_date":"h2", ...}},
{"order_108": {"items":"i1", "delivery_date":"i2", ...}},
{"order_109": {"items":"j1", "delivery_date":"j2", ...}},
{"order_110": {"items":"k1", "delivery_date":"k2", ...}}
```

Source: CMU 11-785 (Spring 2024), Lecture 19: Transformers and Newer Architectures, slide 39.

Query, Key, Value (QKV)



Source: CMU 11-785 (Spring 2024), Lecture 19: Transformers and Newer Architectures, slide 40.

Query, Key, Value (QKV)

{Query: “Order details of order_104”}

OR

{Query: “Order details of order_106”}


```
{"order_100": {"items":"a1", "delivery_date":"a2", ...}},
{"order_101": {"items":"b1", "delivery_date":"b2", ...}},
{"order_102": {"items":"c1", "delivery_date":"c2", ...}},
{"order_103": {"items":"d1", "delivery_date":"d2", ...}},
{"order_104": {"items":"e1", "delivery_date":"e2", ...}},
{"order_105": {"items":"f1", "delivery_date":"f2", ...}},
{"order_106": {"items":"g1", "delivery_date":"g2", ...}},
{"order_107": {"items":"h1", "delivery_date":"h2", ...}},
{"order_108": {"items":"i1", "delivery_date":"i2", ...}},
{"order_109": {"items":"j1", "delivery_date":"j2", ...}},
{"order_110": {"items":"k1", "delivery_date":"k2", ...}}
```


Query

1. Search for info

Key

1. Interacts directly with Queries
2. Distinguishes one object from another
3. Identify which object is the most relevant and by how much

Value

1. Actual details of the object
2. More fine grained

Source: CMU 11-785 (Spring 2024), Lecture 19: Transformers and Newer Architectures, slide 41.

Attention-Based Decoding – Overview

- The decoder uses attention to select relevant encoder hidden states at each decoding step.

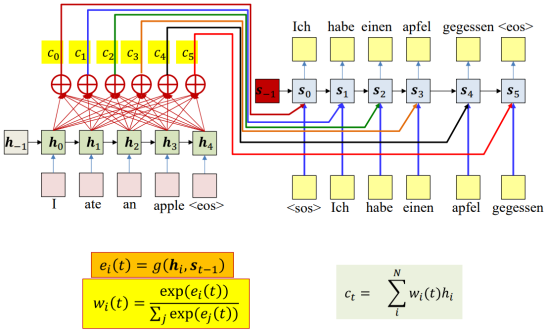
Combines:

- Previous decoder state
- Encoder hidden states
- Attention context vector

- The output at time t:

$$y_t = \text{Decoder}(y_{t-1}, \text{context}_t)$$

Attention-Based Decoding

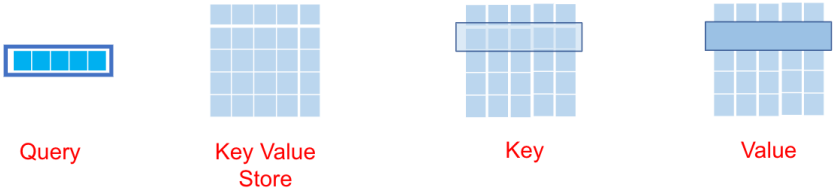


Source: after CMU 11-785 (Spring 2024), Lecture 18: Sequence to Sequence Models & Attention, slides 28 and 31.

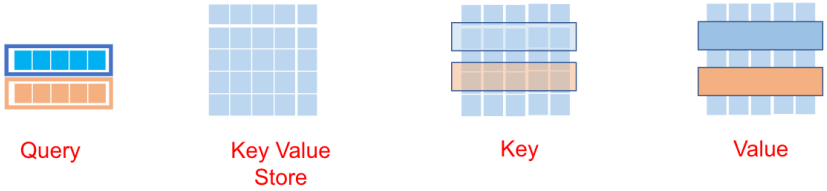
- Every encoder state h_i is scored against the previous decoder state, $e_i(t) = g(h_i, s_{t-1})$, and the scores are softmaxed into weights $w_i(t)$.
- Those weights already form a distribution, so the context is the plain weighted sum $c_t = \sum_i w_i(t)\, h_i$.

Decoding - Steps

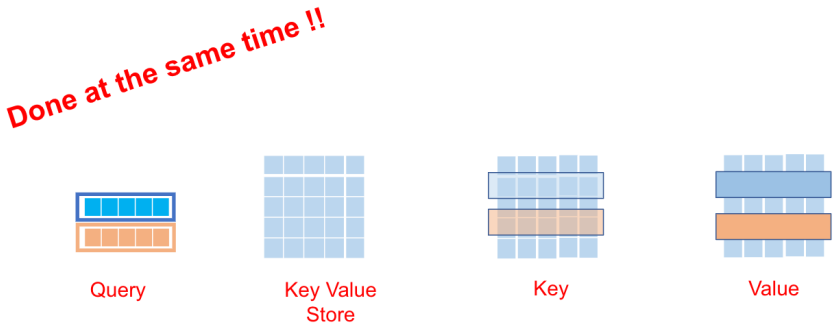
1. **Compute query**: Generate a query vector from the current decoder hidden state.
2. **Align with encoder outputs**: Compare the query with encoder outputs (keys) to measure relevance.
3. **Get attention weights**: Apply softmax to the alignment scores to obtain attention weights.
4. **Compute context**: Calculate the context vector as the weighted sum of encoder outputs using the attention weights.
5. **Feed into decoder**: Use the context vector and previous outputs as input for the next decoding step.



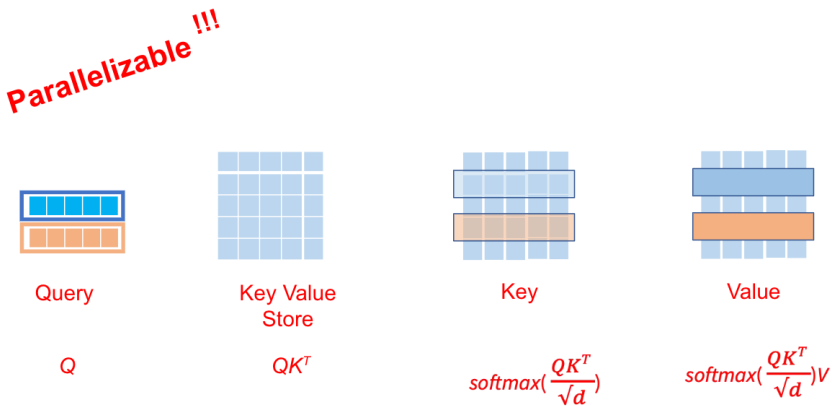
Source: CMU 11-785 (Spring 2024), Lecture 19: Transformers and Newer Architectures, slide 42.



Source: CMU 11-785 (Spring 2024), Lecture 19: Transformers and Newer Architectures, slide 43.



Source: CMU 11-785 (Spring 2024), Lecture 19: Transformers and Newer Architectures, slide 44.



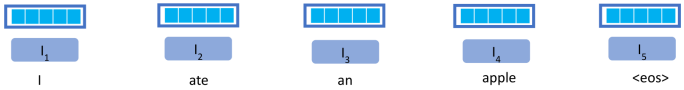
Source: CMU 11-785 (Spring 2024), Lecture 19: Transformers and Newer Architectures, slide 45.

Self-Attention and Cross-Attention

The same scoring machinery serves two arrangements, distinguished only by where **Q**, **K** and **V** are projected from.

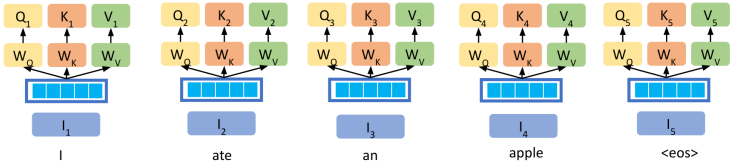
- **Cross-attention:** the query comes from the decoder state and the keys and values from the encoder states, so the decoder reads the source sentence. This is the arrangement used so far.
- **Self-attention:** query, key and value are all projected from one and the same sequence, so every token is re-expressed as a weighted sum of its neighbours.

The next slides trace self-attention over the input "I ate an apple <eos>", one score at a time. No token's output depends on another token's output, so every position can be computed at once: this is what makes the mechanism parallelizable.

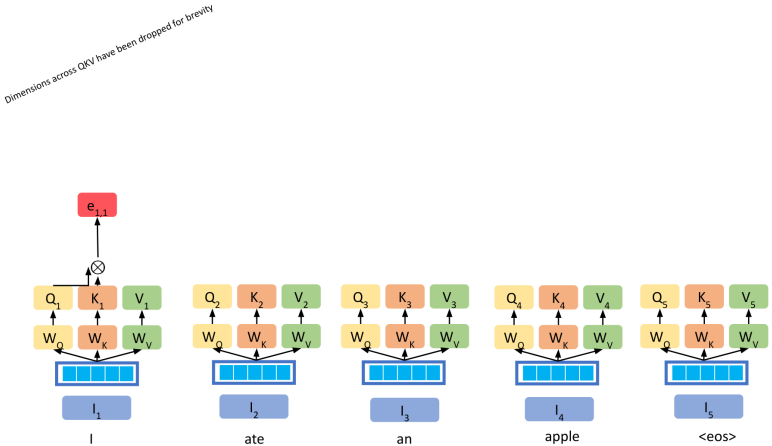


Source: CMU 11-785 (Spring 2024), Lecture 19: Transformers and Newer Architectures, slide 46.

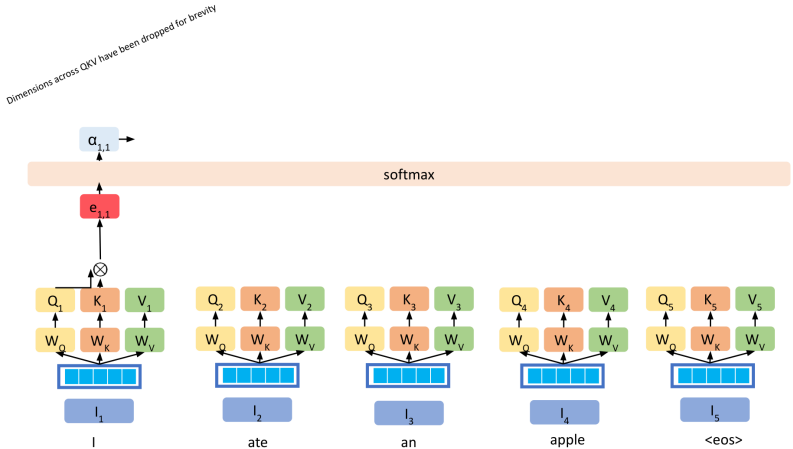
Attention



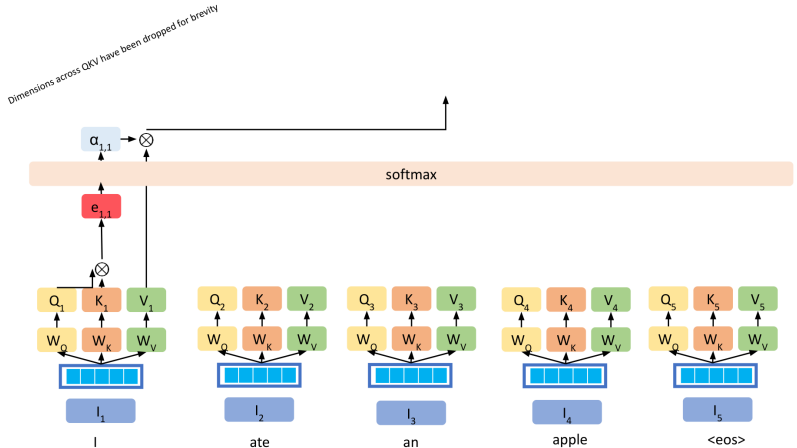
Source: CMU 11-785 (Spring 2024), Lecture 19: Transformers and Newer Architectures, slide 47.



Source: CMU 11-785 (Spring 2024), Lecture 19: Transformers and Newer Architectures, slide 48.

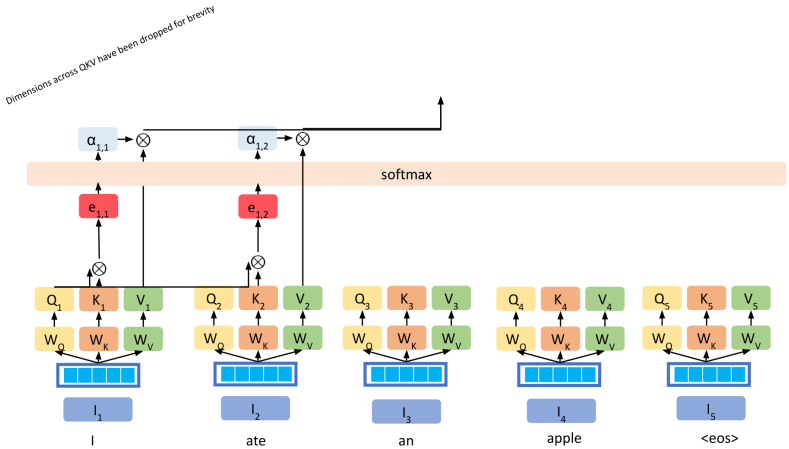


Source: CMU 11-785 (Spring 2024), Lecture 19: Transformers and Newer Architectures, slide 49.

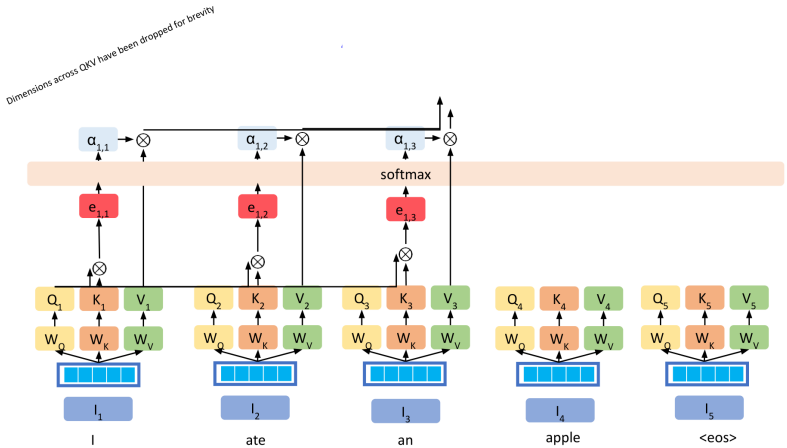


Source: CMU 11-785 (Spring 2024), Lecture 19: Transformers and Newer Architectures, slide 50.

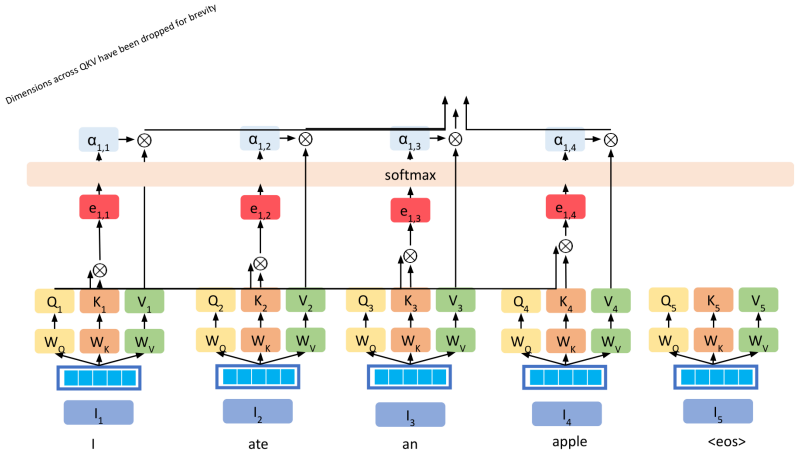
Attention



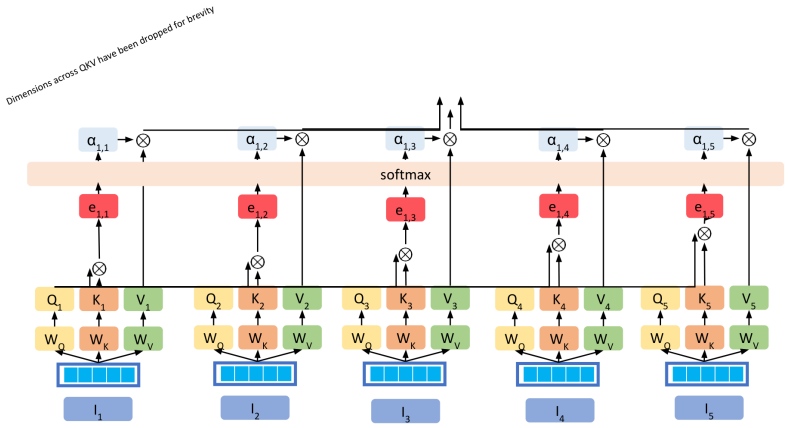
Source: CMU 11-785 (Spring 2024), Lecture 19: Transformers and Newer Architectures, slide 51.



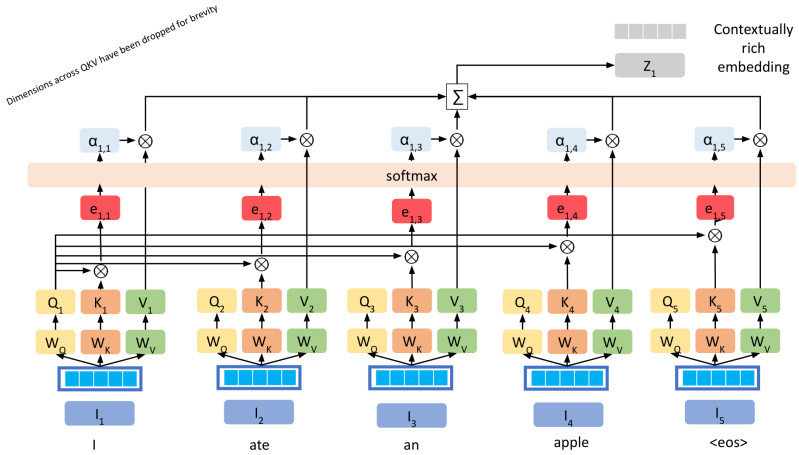
Source: CMU 11-785 (Spring 2024), Lecture 19: Transformers and Newer Architectures, slide 52.



Source: CMU 11-785 (Spring 2024), Lecture 19: Transformers and Newer Architectures, slide 53.

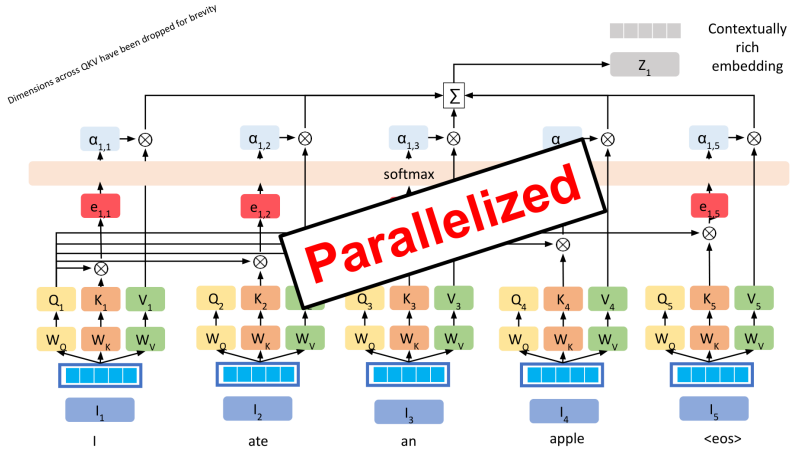


Source: CMU 11-785 (Spring 2024), Lecture 19: Transformers and Newer Architectures, slide 54.



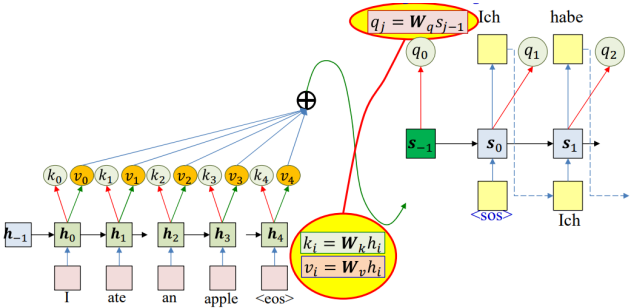
Source: CMU 11-785 (Spring 2024), Lecture 19: Transformers and Newer Architectures, slide 55.

Attention



Source: CMU 11-785 (Spring 2024), Lecture 19: Transformers and Newer Architectures, slide 56.

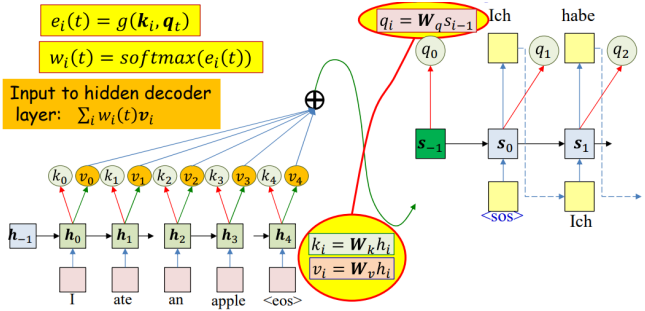
Attention



- Encoder outputs an explicit "key" and "value" at each input time
  - Key is used to evaluate the importance of the input at that time, for a given output
- Decoder outputs an explicit "query" at each output time
  - Query is used to evaluate which inputs to pay attention to
- The weight is a function of key and query
- The actual context is a weighted sum of value

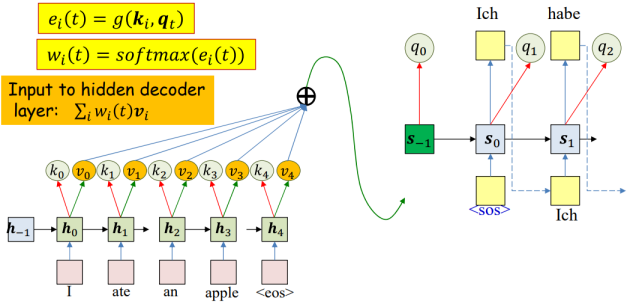
Source: CMU 11-785 (Spring 2024), Lecture 18: Sequence to Sequence Models & Attention, slide 58.

Attention



- Encoder outputs an explicit "key" and "value" at each input time
  - Key is used to evaluate the importance of the input at that time, for a given output
- Decoder outputs an explicit "query" at each output time
  - Query is used to evaluate which inputs to pay attention to
- The weight is a function of key and query
- The actual context is a weighted sum of value

Source: CMU 11-785 (Spring 2024), Lecture 18: Sequence to Sequence Models & Attention, slide 59.



Special case: $k_i = v_i = h_i$

$q_t = s_{t-1}$

We will continue using this assumption in the following slides but in practice the query-key-value format is used

Source: CMU 11-785 (Spring 2024), Lecture 18: Sequence to Sequence Models & Attention, slide 60.

Attention

```
# Assuming encoded input
#   (K,V) = [k_enc[0]… k_enc[T]], [v_enc[0]… v_enc[T]]
# is available

t = -1
h_out[-1] = 0    # Initial Decoder hidden state
q[0] = 0         # Initial query

# Note: begins with a "start of sentence" symbol
#  <sos> and <eos> may be identical
Y_out[0] = <sos>
do
    t = t+1
    C  = compute_context_with_attention(q[t], K, V)
    y[t],h_out[t],q[t+1] = RNN_decode_step(h_out[t-1], y_out[t-1], C)
    y_out[t] = generate(y[t]) # Random, or greedy
until y_out[t] == <eos>
```

Source: CMU 11-785 (Spring 2024), Lecture 18: Sequence to Sequence Models & Attention, slide 61.

Decoding – Example: Translation

Example: English to French Translation

Source Sentence (English)
The cat sat on the mat.

Target Sentence (French)
Le chat s'est assis sur le tapis.

- At each decoding step, the decoder generates a French word by:
  1. Using the current decoder hidden state.
  2. Attending to relevant English words via attention weights.
  3. Computing a context vector as a weighted sum of encoder outputs.
- For example, when generating "*chat*", the attention focuses on "cat".
- The process repeats for each word, dynamically shifting attention.

Limitations of Attention Mechanisms

- **Quadratic Complexity:** Attention computation scales as $\mathcal{O}(n^2)$ with sequence length, making it expensive for long sequences.
- **Shallow Reasoning:** Standard attention mechanisms typically consider only one step of interaction, limiting deep reasoning capabilities.
- **Noisy Attention Maps:** Attention distributions can be diffuse or hard to interpret, making model decisions less transparent.
- **Limited Long-Term Memory:** Attention alone may struggle to capture dependencies over very long contexts.

Future Directions in Attention Mechanisms

- **Approximate and Sparse Attention:**
  - *Linformer, Performer, Longformer, Routing Transformer* – Replace the dense score matrix with a low-rank, kernelized or sparse stand-in, trading exactness for sub-quadratic cost.
- **Exact but IO-Aware Attention:**
  - *FlashAttention* – Computes the same attention, tiled to cut traffic to GPU memory. It saves wall-clock time rather than arithmetic.
- **Sparse Normalization:**
  - *Sparsemax* – A softmax replacement that can assign exactly zero weight, giving sharper and more interpretable attention maps.
- **Cross-modal Attention:**
  - *Flamingo, BLIP-2* – Cross-attention layers let a language model read the output of a vision encoder.
- **Learnable Routing:**
  - *Mixture-of-Experts, Mixture-of-Depths* – A learned gate sends each token to a subset of the parameters or layers.

Summary

Concept	Key Takeaway
Alignment	Scores relevance between states
QKV	Core mechanism to compute attention
Decoding	Uses attention to generate outputs
Limitation	Costly, hard to scale to long seqs
Future	Efficient, structured, multimodal attention

References

These slides have been adapted from:

- Younes Mourri & Lukasz Kaiser, Natural Language Processing Specialization. DeepLearning.AI
- Bhiksha Raj & Rita Singh, 11-785 Introduction to Deep Learning, CMU (Spring 2024), Lecture 18: *Sequence to Sequence Models & Attention* and Lecture 19: *Transformers and Newer Architectures*
- Sebastian Raschka, Build a Large Language Model (From Scratch), Manning, 2024 (Ch. 3 multi-head attention figure)

Figure credits give the source lecture and slide number. Those marked "after" are adapted rather than reproduced verbatim: in the Lecture 18 diagrams the context-vector notation has been corrected to match the definitions used here, and the Raschka figure has been re-annotated.

References

- Bahdanau, D., Cho, K., & Bengio, Y. (2015). *Neural Machine Translation by Jointly Learning to Align and Translate.*
- Luong, M.-T., Pham, H., & Manning, C. D. (2015). *Effective Approaches to Attention-based Neural Machine Translation.* EMNLP.
- Vaswani, A., Shazeer, N., Parmar, N., et al. (2017). *Attention Is All You Need.*
- Raffel, C., Shazeer, N., Roberts, A., et al. (2020). *Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer.*
- Choromanski, K., Likhosherstov, V., Dohan, D., et al. (2021). *Rethinking Attention with Performers.*
- Tay, Y., Dehghani, M., Bahri, D., & Metzler, D. (2022). *Efficient Transformers: A Survey.*
- Rush, A. (2018). *The Annotated Transformer.* Harvard NLP.